

HACETTEPE UNIVERSITY
FACULTY OF ENGINEERING

DEPARTMENT OF ARTIFICIAL INTELLIGENCE ENGINEERING

**Toxic Comment Classification: Fine-Tuned BERT vs.
Local LLM Prompting**

AIN 428 – Information Retrieval
FINAL PROJECT REPORT

Prepared by:
Yusuf Emir Cömert - 2220765023

Date:
June 2026

Table of Contents

1. Abstract	3
2. Introduction	3
3. Problem Statement & Dataset	4
3.1 Jigsaw Toxic Comment Classification Challenge	4
3.2 Data Preprocessing & Stratified Sampling	5
3.3 Cross-Validation Setup	5
4. Methodology	6
4.1 Method A: Fine-Tuned DistilBERT	6
4.2 Method B: Local LLM Prompting (Zero-Shot & Few-Shot)	7
4.3 Evaluation Metrics	8
5. Experimental Results	8
5.1 DistilBERT Results	9
5.2 LLM Zero-Shot Results	9
5.3 LLM Few-Shot Results	10
5.4 Per-Label Analysis	10
6. Discussion & Benchmarking	12
6.1 Why Fine-Tuning Outperforms Prompting	12
6.2 The LLM Over-Prediction Problem	13
6.3 Rare Class Failure	13
6.4 Compute Cost Comparison	14
6.5 Study Limitations	15
7. Conclusion	15
8. References	17

1. Abstract

In this final project, the problem of automated toxic comment classification is addressed using two fundamentally different computational paradigms. The primary objective is to conduct a rigorous comparative analysis between **supervised fine-tuning of DistilBERT** and **local LLM prompting** using the SmolLM2-1.7B-Instruct model, evaluated on the Jigsaw Toxic Comment Classification Challenge dataset.

The Jigsaw dataset contains approximately 159,571 comments labeled across six independent binary toxicity categories. Due to the multi-label nature of the task and severe class imbalance, a 25,000-row stratified subsample was constructed and evaluated using 5-fold multilabel stratified cross-validation with shared splits across both methods.

Experimental results demonstrate a clear performance gap between the two approaches. The fine-tuned DistilBERT model achieved a **macro F1 of 0.424 ± 0.030** and a micro F1 of 0.763 ± 0.006 , significantly outperforming the local LLM under zero-shot (macro F1: 0.141) and few-shot (macro F1: 0.175) settings. The LLM consistently exhibited high recall but extremely low precision, indicating a systematic over-prediction of toxic labels. Both methods struggle on rare categories such as *threat* and *identity_hate*. This report presents the problem formulation, preprocessing pipeline, model architectures, evaluation methodology, and a detailed discussion of results.

2. Introduction

Online platforms and social media ecosystems generate enormous volumes of user-generated content every day. Within this content, toxic, abusive, and harmful language poses a serious challenge for platform moderation. Manual review at scale is infeasible, necessitating automated approaches capable of identifying and flagging toxic content with high accuracy. The challenge is further complicated by the multi-label nature of toxicity: a single comment may simultaneously be *toxic*, *obscene*, and *insulting*, requiring systems to produce nuanced, multi-dimensional predictions rather than simple binary outputs.

Recent advances in Natural Language Processing (NLP) have produced two dominant paradigms for tackling such tasks. The first is **task-specific fine-tuning** of pre-trained transformer models such as BERT and its derivatives, which adapts model weights directly to the target label space using labeled training data. The second is **zero-shot or few-shot**

prompting of large language models (LLMs), which leverages the general language understanding of pre-trained models to perform classification without any weight updates, guided only by natural language instructions.

This study investigates both paradigms on consumer GPU hardware (NVIDIA RTX 5060, 8 GB VRAM), asking a concrete research question: *On limited hardware, does fine-tuning a compact transformer outperform prompting a small local LLM without task-specific training?* The answer has practical implications for platform developers who must balance annotation cost, compute budget, and deployment complexity.

The research was conducted following a systematic pipeline consisting of four main stages:

1. **Data Analysis & Preprocessing:** The Jigsaw dataset was analyzed using label distribution statistics, and a stratified subsample with shared cross-validation folds was constructed to ensure fair comparison.
2. **Fine-Tuned BERT (Method A):** A DistilBERT model was fine-tuned end-to-end on the multi-label classification task using a sigmoid output layer and binary cross-entropy loss, trained with early stopping over three epochs per fold.
3. **LLM Prompting (Method B):** The SmolLM2-1.7B-Instruct model was prompted in zero-shot and few-shot configurations to classify comments into the six toxicity categories, producing structured JSON outputs.
4. **Evaluation & Comparison:** Both methods were evaluated on identical held-out test folds using macro and micro F1 scores, with per-label breakdowns to identify class-specific failure modes.

This report details the technical implementation, the design choices made for each method, and a thorough analysis of the results, including the limitations inherent to both approaches.

3. Problem Statement & Dataset

3.1 Jigsaw Toxic Comment Classification Challenge

The dataset used in this study originates from the **Jigsaw Toxic Comment Classification Challenge** hosted on Kaggle. It contains ~159,571 Wikipedia talk page comments, each annotated by human raters across six independent binary toxicity labels:

Label	Description	Count	Prevalence
toxic	Rude, disrespectful, likely to drive users away	15,294	9.58%
severe_toxic	Extremely hateful or aggressive content	1,595	1.00%
obscene	Obscenities or vulgar language	8,449	5.29%
threat	Direct threat of violence	478	0.30%
insult	Insulting or inflammatory language	7,877	4.93%
identity_hate	Targets identity (race, religion, gender, etc.)	1,405	0.88%
Clean (no label)	Comments with no toxic label	143,346	89.83%

This is a **multi-label classification** problem: a comment can belong to multiple toxicity categories simultaneously, or to none at all. Approximately **16,225 comments (10.2%)** carry at least one toxic label. The dataset exhibits **severe class imbalance**, with the rarest category (*threat*, 0.30%) being approximately 32 times less frequent than the most common (*toxic*, 9.58%). This imbalance has profound implications for model training, threshold selection, and the interpretation of evaluation metrics.

3.2 Data Preprocessing & Stratified Sampling

Given the computational constraints of running both a fine-tuning pipeline and an LLM inference loop on a consumer GPU, working with the full 159K-row dataset was impractical. To address this, a **25,000-row stratified subsample** was drawn from the full training set using **MultilabelStratifiedShuffleSplit** from the *iterative-stratification* library (seed = 42). This method ensures that the multi-label distribution of the original dataset is preserved in the subsample, preventing accidental over- or under-representation of rare toxic classes.

The subsample was saved to *data/train_subsampled.csv*. Both BERT and LLM experiments operate exclusively on this subsample, ensuring that any performance differences observed between methods cannot be attributed to differences in training data.

3.3 Cross-Validation Setup

A **5-fold multilabel stratified cross-validation** scheme was implemented using **MultilabelStratifiedKFold** (seed = 42). The fold indices were precomputed and stored as

NumPy arrays in *data/fold_indices/* alongside a *metadata.json* file documenting the split configuration.

The critical design decision here is that **both the fine-tuned BERT model and the LLM prompting experiments use the exact same fold indices**. This guarantees that any comparison of results is conducted on identical data partitions, eliminating data split variance as a confounding factor. Per fold, the data is distributed as follows:

- **Train:** 16,000 comments (used for BERT fine-tuning only)
- **Validation:** 4,000 comments (used for BERT early stopping only)
- **Test:** 5,000 comments (held-out evaluation set for both methods)

LLM responses for each fold and mode (zero-shot, few-shot) were **cached to disk** under *results/llm/cache/*, enabling interrupted runs to be resumed without re-running inference. This caching mechanism was essential given the slow inference speed of the 1.7B LLM (several hours per full cross-validation run).

4. Methodology

Two distinct computational frameworks were developed to address the multi-label toxic comment classification task. The first is a supervised learning approach that adapts a pre-trained transformer to the target label space. The second is a training-free approach that relies entirely on the general language understanding of a small instruction-tuned LLM.

4.1 Method A: Fine-Tuned DistilBERT

The fine-tuning approach is built on **DistilBERT-base-uncased**, a distilled variant of BERT that retains approximately 97% of BERT’s language understanding performance while using 40% fewer parameters, making it well-suited for GPU-constrained environments. A multi-label classification head (a linear layer followed by a sigmoid activation) was added on top of the [CLS] token representation, producing six independent probability scores — one per toxicity category.

The model was trained end-to-end with the following configuration:

Hyperparameter	Value
Base model	distilbert-base-uncased (local copy)
Task head	Multi-label classification (sigmoid + BCE loss)
Max sequence length	256 tokens
Training epochs	3 (with early stopping, patience = 1)
Batch size	16
Learning rate	2e-5
Classification threshold	0.5 on sigmoid output
Mixed precision	FP16 on GPU (CUDA 12.8)

Each of the five folds was trained independently. The model checkpoint with the lowest **validation loss** was retained and used for evaluation on the held-out test fold. Checkpoints were saved under *results/bert/checkpoints/* and excluded from version control due to their large size. Binary cross-entropy loss was used as the training objective, treating each label as an independent binary classification problem.

An important design consideration is the choice of **threshold**. A uniform threshold of 0.5 was applied to all six sigmoid outputs. While per-label threshold tuning could improve macro F1 — particularly for rare classes — this was deliberately left as a fixed value to maintain simplicity and to isolate model-level performance from threshold engineering.

4.2 Method B: Local LLM Prompting (Zero-Shot & Few-Shot)

The prompting approach uses **HuggingFaceTB/SmolLM2-1.7B-Instruct**, a small instruction-tuned language model that can be loaded on a consumer GPU with 8 GB VRAM. The model was loaded in FP16 precision and used with greedy decoding (temperature = 0.0) to ensure deterministic and reproducible outputs. The model was instructed to classify each comment into the six toxicity categories by producing a structured **JSON object** with six binary fields.

Two prompting configurations were evaluated:

- **Zero-Shot:** The prompt contains label definitions, a JSON output schema, and the comment text. No examples are provided. The model must infer classification boundaries from the label descriptions alone.
- **Few-Shot:** The prompt additionally includes three in-context examples covering distinct categories (an insult, a benign comment, and a threat). These examples are

hardcoded in the prompt template and included before the target comment to provide concrete classification guidance.

A **regex-based JSON parser** with a fallback mechanism was implemented to handle malformed outputs, as the 1.7B model occasionally produced incomplete or syntactically invalid JSON responses. The model was unloaded from GPU memory between folds to reduce VRAM pressure during long multi-fold runs.

Setting	Value
Model	HuggingFaceTB/SmolLM2-1.7B-Instruct
Temperature	0.0 (greedy decoding)
Max new tokens	64
Output format	JSON with six binary (0/1) fields
Zero-shot context	Label definitions + JSON schema
Few-shot context	3 in-context examples (insult, benign, threat)

4.3 Evaluation Metrics

Both methods were evaluated using standard multi-label classification metrics computed with scikit-learn. The primary metric is **macro F1**, which computes the unweighted mean F1 score across all six labels. Macro F1 treats each class equally regardless of frequency, making it the most informative metric for imbalanced multi-label tasks where rare-class performance matters. As a secondary metric, **micro F1** is reported, which pools all label-level true positives, false positives, and false negatives before computing F1, thus giving more weight to frequent labels.

Per-label **F1** is reported in Table 6 to identify class-specific failure modes; precision and recall are discussed qualitatively in Section 6. All metrics are reported as mean $\pm$ standard deviation across the five held-out test folds.

5. Experimental Results

This section presents the quantitative performance of both methods across all five cross-validation folds. Per-fold breakdowns, overall summary statistics, and per-label analyses are provided.

5.1 DistilBERT Results

The fine-tuned DistilBERT model was evaluated on the held-out test fold of each cross-validation split. The results demonstrate consistent performance across folds, with relatively low standard deviation indicating stable generalization.

Fold	Macro F1	Micro F1
0	0.401	0.767
1	0.466	0.768
2	0.443	0.757
3	0.416	0.766
4	0.395	0.755
Mean $\pm$ Std	0.424 $\pm$ 0.030	0.763 $\pm$ 0.006

The per-label breakdown reveals a clear bimodal pattern in performance. The model achieves strong results on the three most frequent labels — *toxic* (F1: 0.82), *obscene* (F1: 0.84), and *insult* (F1: 0.74) — but completely fails to detect *threat* (F1: 0.00 across all folds) and barely detects *identity_hate* (F1: 0.02). The *severe_toxic* label falls in an intermediate zone with an F1 of 0.13, suggesting partial learning but insufficient training signal.

The complete failure on *threat* is particularly striking. With only 478 instances in the full dataset (~ 75 in the 25k subsample, ~ 15 per test fold), the model never learns to produce confident predictions above the 0.5 threshold. This is a direct consequence of using a fixed, uniform threshold combined with severe class imbalance.

5.2 LLM Zero-Shot Results

The zero-shot LLM configuration yields substantially lower performance compared to fine-tuned DistilBERT. Crucially, performance is remarkably consistent across folds (standard deviation < 0.002), reflecting the fact that the LLM receives no training signal and its outputs depend only on the prompt structure and the model’s pre-trained weights.

Fold	Macro F1	Micro F1
0	0.138	0.154
1	0.140	0.155
2	0.142	0.157

Fold	Macro F1	Micro F1
3	0.142	0.157
4	0.141	0.156
Mean $\pm$ Std	0.141 ± 0.001	0.156 ± 0.001

The primary failure mode in zero-shot prompting is **systematic over-prediction**. The model assigns multiple toxic labels to a large proportion of comments, achieving recall values above 0.80 on most categories but precision below 0.15. This behavior likely stems from the model’s tendency to err on the side of caution when presented with ambiguous content, combined with the absence of calibrated probability estimates in a greedy decoding setting.

5.3 LLM Few-Shot Results

Incorporating three in-context examples improves macro F1 by approximately 0.035 over zero-shot, representing a relative improvement of about 25%. The few-shot configuration achieves a mean macro F1 of **0.175 ± 0.002** and a micro F1 of 0.171 ± 0.001 .

Fold	Macro F1	Micro F1
0	0.175	0.169
1	0.178	0.173
2	0.175	0.171
3	0.174	0.170
4	0.175	0.170
Mean $\pm$ Std	0.175 ± 0.002	0.171 ± 0.001

The few-shot improvement is not uniform across labels. The largest gains are observed on *obscene* (zero-shot F1: 0.27 $\rightarrow$ few-shot F1: 0.47) and *identity_hate* (0.06 $\rightarrow$ 0.11). The in-context examples appear to provide useful calibration for these specific categories. However, performance on *insult* actually decreases slightly (0.26 $\rightarrow$ 0.21), possibly due to the examples shifting the model’s threshold in a suboptimal direction for that label.

5.4 Per-Label Analysis

The following table presents the mean F1 score per label across all five folds for each method. This breakdown reveals the structural differences in how each approach handles the multi-label task.

Label	DistilBERT F1	LLM Zero-Shot F1	LLM Few-Shot F1
toxic	0.82	0.18	0.20
obscene	0.84	0.27	0.47
insult	0.74	0.26	0.21
severe_toxic	0.13	0.06	0.05
threat	0.00	0.02	0.03
identity_hate	0.02	0.06	0.11

The per-label data highlights an important asymmetry: for rare labels (*threat*, *identity_hate*), the LLM’s over-prediction tendency paradoxically produces *higher* F1 than DistilBERT’s conservative predictions, because at least some true positives are captured. DistilBERT’s zero F1 on *threat* means it never predicts this label at all, reflecting the challenge of learning from fewer than *~60 positive training examples per fold* without class-weighting strategies.

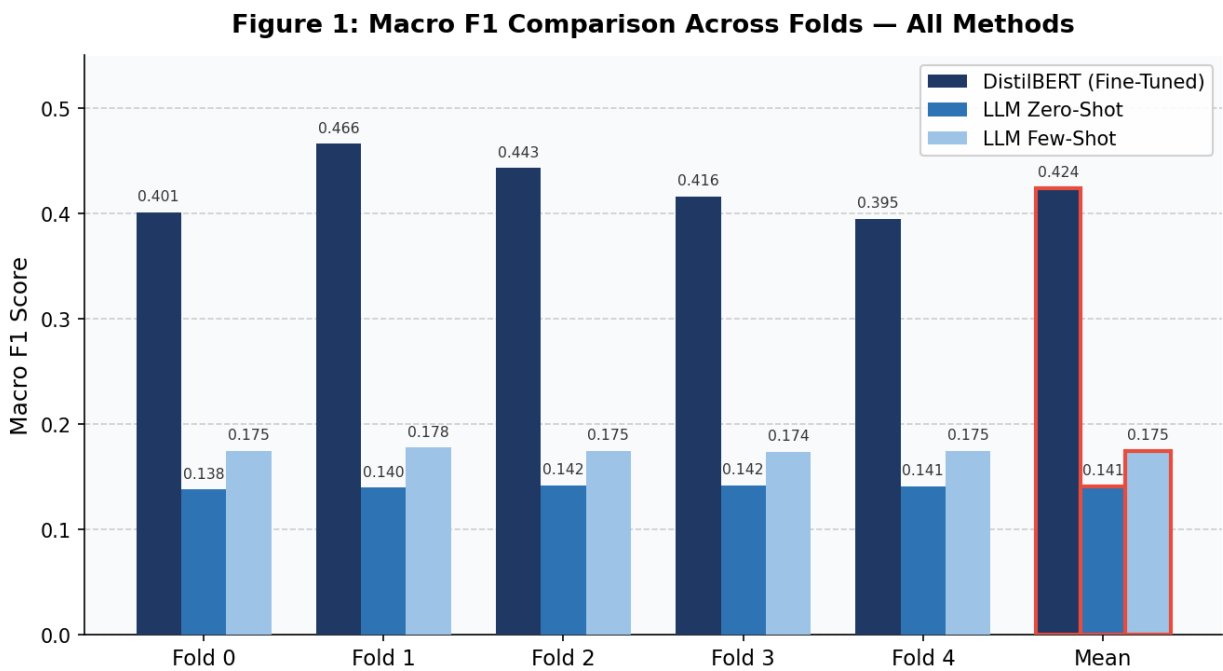


Figure 1: Macro F1 scores across all five folds for DistilBERT, LLM zero-shot, and LLM few-shot. The mean fold (rightmost group) is outlined in red.

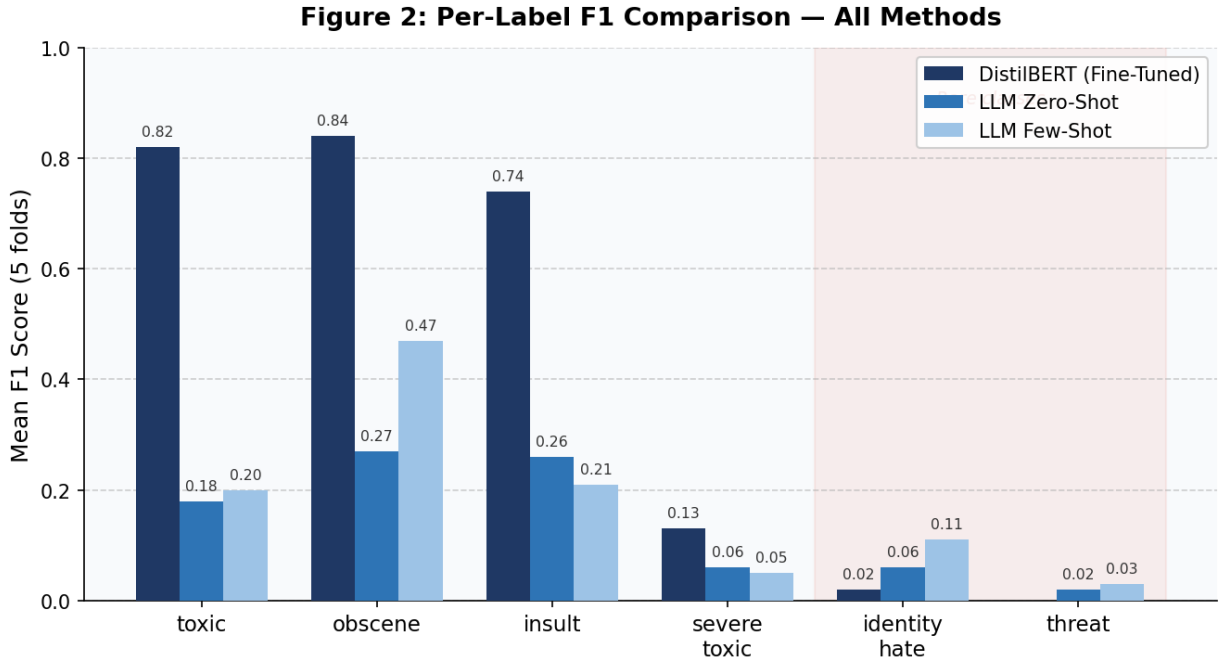


Figure 2: Per-label mean F1 across all five folds. The shaded region marks the two rarest classes (*identity_hate*, *threat*) where both methods struggle.

6. Discussion & Benchmarking

6.1 Why Fine-Tuning Outperforms Prompting

The performance gap between fine-tuned DistilBERT and the prompted SmolLM2-1.7B-Instruct is substantial and consistent. Fine-tuning adapts the model’s internal representations directly to the Jigsaw label space using thousands of labeled examples per fold. The gradient updates during training shift the model’s decision boundaries to align with the specific linguistic patterns associated with each toxicity category in this dataset. The resulting model is calibrated: its sigmoid outputs reflect the statistical patterns of the training data, not just the model’s generic sense of what “toxic” means.

In contrast, the 1.7B LLM must infer classification boundaries from a short prompt describing the label definitions. Without access to the training data’s statistical patterns, it falls back on its pre-trained sense of toxicity, which does not align with the annotators’ specific criteria. Furthermore, the LLM lacks calibrated probability estimates; its greedy decoding produces hard binary decisions without the nuanced confidence weighting that a sigmoid threshold provides. This leads directly to the systematic over-prediction observed in the results.

6.2 The LLM Over-Prediction Problem

The LLM’s tendency to assign multiple toxic labels indiscriminately is a consistent and significant failure mode. Across both zero-shot and few-shot configurations, recall values exceed 0.80 on most labels, but precision falls below 0.15. This precision-recall imbalance yields a low F1 despite the high recall, because the metric penalizes false positives and false negatives symmetrically.

Several factors likely contribute to this behavior. First, the model is small (1.7B parameters), and small models tend to be poorly calibrated, often defaulting to conservative or overly inclusive predictions when uncertain. Second, the JSON output format introduces structural pressure: the model is prompted to produce a complete six-field object, which may encourage it to assign positive values to avoid producing an all-zero response that might seem “wrong.” Third, the absence of negative examples in the zero-shot setting provides no explicit signal for what does *not* constitute each toxicity category.

The few-shot setting partially addresses the third issue by including one benign example alongside the toxic ones, which helps the model understand that “0” outputs are valid. This explains the modest but consistent improvement from zero-shot to few-shot, particularly on categories directly represented by the in-context examples.

6.3 Rare Class Failure

Both methods struggle on *threat* and *identity_hate*, the two rarest labels in the dataset. For DistilBERT, the failure on *threat* is total: zero F1 across all five folds. This suggests that with fewer than ~75 positive training examples per fold and a uniform 0.5 threshold, the model’s sigmoid output for this label never exceeds the classification threshold. The gradient signal from 75 positive examples is simply overwhelmed by the signal from ~15,925 negative examples in the training fold.

Addressing rare class failure would require targeted interventions, including:

- **Class-weighted loss:** Assigning higher loss weights to rare class positive examples to amplify their gradient signal during training.
- **Per-label threshold tuning:** Optimizing the classification threshold independently for each label on the validation fold, rather than using a fixed 0.5 threshold across all labels.

- **Oversampling or data augmentation:** Increasing the representation of rare-class examples through oversampling or generation-based augmentation techniques.

The LLM’s over-prediction, while producing low precision, at least results in non-zero recall for these rare classes. This illustrates a scenario where the LLM’s lack of calibration inadvertently works in its favor: by predicting broadly, it captures some true positives that DistilBERT completely misses.

6.4 Compute Cost Comparison

Beyond predictive performance, the two approaches differ substantially in their computational demands. The following table summarizes the practical cost dimensions:

Aspect	DistilBERT (Fine-Tuned)	SmolLM2-1.7B (Prompted)
Training required	Yes (~30 min/fold on RTX 5060)	No
Inference speed	Fast (batched GPU inference)	Very slow (sequential, token-by-token)
Total experiment time	~2.5 hours (5 folds)	Several hours per mode
Adaptability to new labels	Requires retraining	Prompt modification only
Accuracy (Macro F1)	0.424	0.141 / 0.175
Hardware requirement	8 GB VRAM (FP16)	8 GB VRAM (FP16)

For production deployment on limited hardware, fine-tuned DistilBERT is the clearly superior choice in terms of both accuracy and inference throughput. The LLM prompting approach, however, retains a distinct advantage for **rapid prototyping**: adding a new toxicity category requires only modifying the prompt definition, whereas the BERT approach requires collecting labeled data and retraining. This tradeoff between adaptability and accuracy is a recurring theme in NLP system design — zero-shot prompt engineering offers speed and flexibility, while supervised fine-tuning delivers precision at the cost of data and compute.

6.5 Study Limitations

Several limitations constrain the scope of the conclusions drawn in this study and should be considered when interpreting the results:

- **25,000-row subsample, not the full dataset:** All experiments were conducted on a 25,000-row stratified subsample of the full 159,571-comment Jigsaw dataset. While the subsample preserves the multi-label distribution, rare-class counts per fold are substantially lower than what training on the full dataset would provide. Results on the full dataset may differ, particularly for the rarest labels.
- **Single small LLM, no API or larger models:** The prompting experiments used only SmolLM2-1.7B-Instruct. Larger local models (7B, 13B) or commercial API models (GPT-4, Claude) would likely exhibit different precision-recall characteristics. The over-prediction failure mode observed here may be specific to the 1.7B scale and should not be generalized to all LLM-based classifiers.
- **Fixed 0.5 threshold for all labels:** A uniform classification threshold of 0.5 was applied to all six sigmoid outputs. Per-label threshold optimization on the validation fold could substantially improve macro F1, especially for rare classes where the model produces low-confidence positive predictions that fall just below the fixed cutoff.
- **Fixed few-shot examples across all folds:** The three in-context examples used in the few-shot configuration are the same across all five folds. Fold-specific or dynamically retrieved examples (e.g., selected based on similarity to the target comment) could improve calibration and may yield higher few-shot performance, though at the cost of added complexity.
- **No class-weighting or oversampling:** The DistilBERT model was trained with standard binary cross-entropy loss without class-weighting or oversampling of minority labels. As a result, the model is effectively optimized for frequent labels, contributing to its complete failure on the rarest category (threat). Introducing weighted loss or focal loss would likely address this directly.

7. Conclusion

This comparative study addressed the challenge of multi-label toxic comment classification using two fundamentally different NLP paradigms, evaluated under rigorous experimental conditions on identical stratified cross-validation splits. Based on the comprehensive experimental analysis, the following conclusions are drawn:

5. **Fine-Tuning Decisively Outperforms Prompting:** The fine-tuned DistilBERT model achieves a macro F1 of 0.424 ± 0.030 and a micro F1 of 0.763 ± 0.006 , compared to macro F1 values of 0.141 (zero-shot) and 0.175 (few-shot) for the prompted SmolLM2-1.7B-Instruct. The performance gap reflects the fundamental advantage of task-specific weight adaptation over general-purpose language understanding for structured classification tasks.
6. **LLM Over-Prediction is a Systematic Failure Mode:** The local LLM consistently assigns toxic labels too broadly, achieving high recall but very low precision across all configurations. This behavior stems from the model's small size, lack of calibration, and absence of task-specific training signal. Few-shot prompting partially mitigates this by providing calibrating examples, but the improvement is insufficient to approach BERT-level performance.
7. **Rare Classes Remain the Central Challenge:** Both methods fail substantially on the rarest labels (threat, identity_hate). DistilBERT's failure is total for threat (F1: 0.00), driven by insufficient positive examples and a fixed classification threshold. Addressing this would require class-weighted loss functions, per-label threshold optimization, or data augmentation strategies.
8. **Practical Recommendation:** For real-world content moderation applications on consumer hardware, fine-tuned DistilBERT is the better choice, delivering substantially higher accuracy at faster inference speeds. LLM prompting remains useful for rapid prototyping when labeled data is unavailable or when new label definitions must be tested quickly without the cost of retraining.

In conclusion, this project demonstrates that while general-purpose language models offer compelling flexibility through prompt engineering, the availability of labeled training data and sufficient compute for fine-tuning produces models that are far better calibrated and far more accurate on structured multi-label classification tasks. Data-driven weight adaptation — even for compact architectures like DistilBERT — remains essential for achieving high-precision automated content moderation at scale.

8. References

1. Jigsaw / Google. "Toxic Comment Classification Challenge." Kaggle, 2018. Available: <https://www.kaggle.com/c/jigsaw-toxic-comment-classification-challenge>
2. Sanh, V., Debut, L., Chaumond, J., and Wolf, T. "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter." arXiv:1910.01108, 2019.
3. HuggingFace. "SmolLM2-1.7B-Instruct." Available: <https://huggingface.co/HuggingFaceTB/SmolLM2-1.7B-Instruct>
4. Sechidis, K., Tsoumakas, G., and Vlahavas, I. "On the stratification of multi-label data." Proceedings of the European Conference on Machine Learning and Principles and Practice of Knowledge Discovery in Databases (ECML PKDD), 2011.
5. Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding." Proceedings of NAACL-HLT, 2019.
6. Brown, T. et al. "Language Models are Few-Shot Learners." Advances in Neural Information Processing Systems 33, 2020.
7. Pedregosa, F. et al. "Scikit-learn: Machine Learning in Python." Journal of Machine Learning Research, vol. 12, pp. 2825-2830, 2011.
8. Y. E. Comert, "Toxic Comment Classification: InfoRetrieval Project Repository," GitHub, 2026. Available: <https://github.com/yusufemircomert/InfoRetrieval>